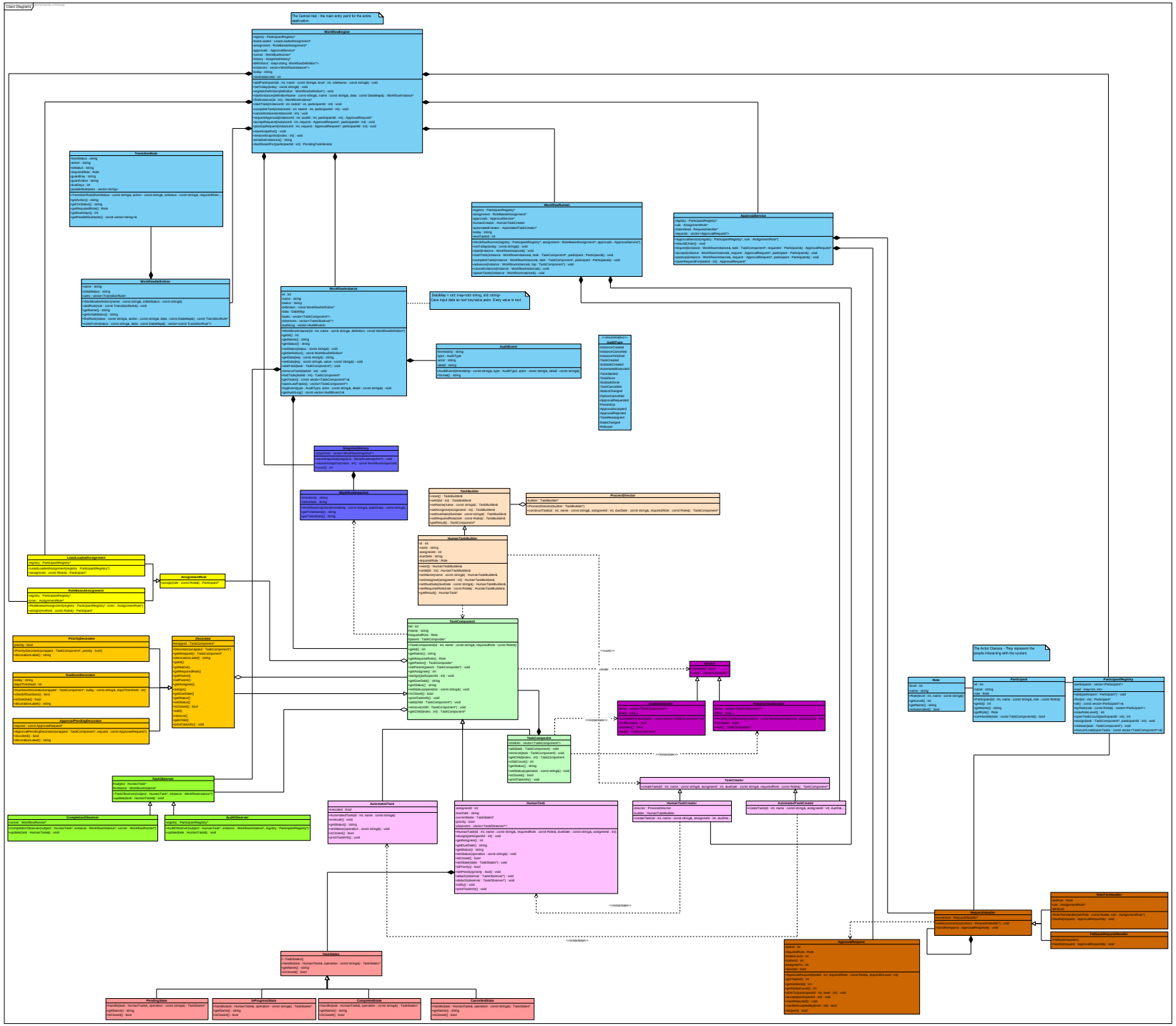
Class Diagram
The Central Hub - the main entry point for the entire workflow.
WorkflowEngine
registry : ParticipantRegistry*
leastLoaded : LeastLoadedAssignment*
roleBased : RoleBasedAssignment*
approvals : ApprovalService*
runner : WorkflowRunner*
history : SnapshotHistory*
definitions : map<string, WorkflowDefinition*>
instances : vector<WorkflowInstance*>
nextId : int
+registerParticipant(id : int, name : const string&, level : int, roleName : const string&) : void
+setTodayBoday : const string&) : void
+defineWorkflow(name : const string&, definition : const WorkflowDefinition&) : void
+startInstance(workflowName : const string&, name : const string&, data : const DataMap&) : WorkflowInstance*
+getInstance(id : int) : WorkflowInstance*
+startTask(instanceId : int, taskId : int, participantId : int) : void
+completeTask(instanceId : int, taskId : int, participantId : int) : void
+cancelInstance(instanceId : int) : void
+requestApproval(instanceId : int, taskId : int, participantId : int) : ApprovalRequest*
+approveRequest(instanceId : int, request : ApprovalRequest*, participantId : int) : void
+rejectApproval(instanceId : int, request : ApprovalRequest*, participantId : int) : void
+getHistory() : SnapshotHistory&
+searchInstances(query : string) : void
+exportInstance(id : int) : string
+getAllInstances() : const vector<WorkflowInstance*>&
+getPendingApprovals(role : int) : void
+getOverdueTasks(instanceId : int) : PendingTaskIterator
TransitionRule
-fromStatus : string
-action : string
-toStatus : string
-requiredRole : Role
-guardKey : string
-guardValue : string
-maxDays : int
-timeoutSubtasks : vector<string>
+TransitionRule(fromStatus : const string&, action : const string&, toStatus : const string&, requiredRole : Role, guardKey : const string& = "", guardValue : const string& = "", maxDays : int = 0, timeoutSubtasks : const vector<string>& = {})
+getAction() : string
+getToStatus() : string
+getRequiredRole() : Role
+getMaxDays() : int
+getTimeoutSubtasks() : const vector<string>&
WorkflowDefinition
-name : string
-initialStatus : string
-rules : vector<TransitionRule>
+WorkflowDefinition(name : const string&, initialStatus : const string&)
+addRule(rule : const TransitionRule&) : void
+getName() : string
+getInitialStatus() : string
+findRule(status : const string&, action : const string&, data : const DataMap&) : const TransitionRule*
+ruleFrom(status : const string&, data : const DataMap&) : vector<const TransitionRule*>
WorkflowInstance
-id : int
-name : string
-status : string
-definition : const WorkflowDefinition*
-data : DataMap
-tasks : vector<TaskComponent*>
-auditLog : vector<AuditEvent>
+WorkflowInstance(id : int, name : const string&, definition : const WorkflowDefinition&)
+getId() : int
+getName() : string
+getStatus() : string
+setStatus(status : const string&) : void
+getDefinition() : const WorkflowDefinition*
+getData(key : const string&) : string
+setData(key : const string&, value : const string&) : void
+addTask(task : TaskComponent*) : void
+removeTask(taskId : int) : void
+findTask(taskId : int) : TaskComponent*
+getTasks() : const vector<TaskComponent*>&
+countLeafTasks() : vector<TaskComponent*>
+logEvent(type : AuditType, actor : const string&, detail : const string&) : void
+getAuditLog() : const vector<AuditEvent>&
DataMap = std::map<std::string, std::string>
Case input data as text key/value pairs. Every value is text.
AuditEvent
-timestamp : string
-type : AuditType
-actor : string
-detail : string
+AuditEvent(timestamp : const string&, type : AuditType, actor : const string&, detail : const string&)
+format() : string
<<enumeration>>
AuditType
InstanceCreated
InstanceStarted
InstanceFinished
TaskCreated
SubtaskCreated
AutomaticExecuted
TaskStarted
TaskDone
SubtaskDone
TaskCancelled
InstanceCancelled
StatusChanged
ApprovalRequested
PendUp
ApprovalAccepted
ApprovalRejected
TaskReassigned
DataChanged
Reload
WorkflowRunner
-registry : ParticipantRegistry*
-assignment : RoleBasedAssignment*
-approvals : ApprovalService*
-humanCreator : HumanTaskCreator
-automatedCreator : AutomatedTaskCreator*
-today : string
-nextTaskId : int
+WorkflowRunner(registry : ParticipantRegistry*, assignment : RoleBasedAssignment*, approvals : ApprovalService*)
+setToday(today : const string&) : void
+getToday() : const string& : void
+startTask(instance : WorkflowInstance&, task : TaskComponent*, participant : Participant&) : void
+completeTask(instance : WorkflowInstance&, task : TaskComponent*, participant : Participant&) : void
+advance(instance : WorkflowInstance&, top : TaskComponent*) : void
+cancelInstance(instance : WorkflowInstance&) : void
+pokeTasks(instance : WorkflowInstance&) : void
ApprovalService
-registry : ParticipantRegistry*
-rule : AssignmentRule*
-channel : RequestHandler*
-requests : vector<ApprovalRequest*>
+ApprovalService(registry : ParticipantRegistry*, rule : AssignmentRule*)
+setRule(rule) : void
+request(instance : WorkflowInstance&, task : TaskComponent*, requester : Participant&) : ApprovalRequest*
+accept(instance : WorkflowInstance&, request : ApprovalRequest*, participant : Participant&) : void
+reject(instance : WorkflowInstance&, request : ApprovalRequest*, participant : Participant&) : void
+openRequestFor(task#) : int[] : ApprovalRequest*
SnapshotHistory
-snapDirs : vector<WorkflowSnapshot*>
+SaveSnapshot(snapshot : WorkflowSnapshot&) : void
+restoreImage(snapDate : int) : const WorkflowSnapShot
+count() : int
WorkflowSnapShot
-snapStamp : string
-snapData : String
+WorkflowSnapShot(snapStamp : const string&, tasksData : const string&)
+getTimestamp() : string
+getTasksData() : string
TaskBuilder
+name() : TaskBuilder&
+setId(id : int) : TaskBuilder&
+setName(name : const string&) : TaskBuilder&
+setAssignee(assignee : int) : TaskBuilder&
+setDueDate(dueDate : const string&) : TaskBuilder&
+setRequiredRole(role : const Role&) : TaskBuilder&
+getResult() : TaskComponent*
ProcessDirector
-builder : TaskBuilder*
+ProcessDirector(builder : TaskBuilder*)
+constructTask(id : int, name : const string&, assigneeId : int, dueDate : const string&, requiredRole : const Role&) : TaskComponent*
HumanTaskBuilder
-id : int
-name : string
-assignee : int
-dueDate : string
-requiredRole : Role
+HumanTaskBuilder()
+setId(id : int) : HumanTaskBuilder&
+setName(name : const string&) : HumanTaskBuilder&
+setAssignee(assigneeId : int) : HumanTaskBuilder&
+setDueDate(dueDate : const string&) : HumanTaskBuilder&
+setRequiredRole(role : const Role&) : HumanTaskBuilder&
+getResult() : HumanTask*
TaskComponent
#id : int
#name : string
#requiredRole : Role
#parent : TaskComposite*
+TaskComponent(id : int, name : const string&, requiredRole : const Role&)
+getId() : int
+getName() : string
+getRequiredRole() : Role
+getParent() : TaskComposite*
+setParent(parent : TaskComposite*) : void
+isAssignedTo(id : int) : bool
+belongsTo(participantId : int) : void
+getDueDate() : string
+getStatus() : string
+setStatus(operation : const string&) : void
+isClosed() : bool
+printTaskInfo() : void
+add(child : TaskComponent*) : void
+remove(child : TaskComponent*) : void
+getChild(index : int) : TaskComponent*
<<use>>
Person
-fullName() : bool
-name() : TaskComponent*
Role
-level : int
-name : string
+Role(level : int, name : const string&)
+getLevel() : int
+getName() : string
+isAtLeast(role) : bool
Participant
-id : int
-name : string
-role : Role
+Participant(id : int, name : const string&, role : const Role&)
+getId() : int
+getName() : string
+getRole() : Role
+canHandle(task : const TaskComponent&) : bool
ParticipantRegistry
-participants : vector<Participant*>
-load : map<int, int>
+add(participant : Participant*) : void
+find(id : int) : Participant*
+all() : const vector<Participant*>&
+byRole(role : const Role&) : vector<Participant*>
+byName(name : const string&) : Participant*
+leastTasks(candidates : const vector<Participant*>&) : Participant*
+assign(task : TaskComponent*, participantId : int) : void
+release(task : TaskComponent*) : void
+resourcesLoad(printTask) : const vector<TaskComponent*>&
The Actor Classes - They represent the people in hierarchy with the system.
PriorityDecorator
-priority : bool
+PriorityDecorator(wrapped : TaskComponent*, priority : bool)
+getAttributeLabel() : string
Decorator
#wrapped : TaskComponent*
+Decorator(wrapped : TaskComponent*)
+getWrapped() : TaskComponent*
+decoratorLabel() : string
+getId()
+getName()
+getRequiredRole()
+getParent()
+setParent()
+isAssignedTo()
+assign()
+getDueDate()
+getStatus()
+setStatus()
+isClosed()
+add()
+remove()
+getChild()
+printTaskInfo() : void
DueSoonDecorator
-today : string
-dayThreshold : int
+DueSoonDecorator(wrapped : TaskComponent*, today : const string&, dayThreshold : int)
+checkIfDueSoon() : bool
+isOverdue() : bool
+decoratorLabel() : string
ApprovalPendingDecorator
-request : const ApprovalRequest*
+ApprovalPendingDecorator(wrapped : TaskComponent*, request : const ApprovalRequest*)
+isJustified() : bool
+decoratorLabel() : string
TaskObserver
#subject : HumanTask*
#instance : WorkflowInstance*
+TaskObserver(subject : HumanTask*, instance : WorkflowInstance*)
+updateState : HumanTaskId : void
CompletionObserver
-runner : WorkflowRunner*
+CompletionObserver(subject : HumanTask*, instance : WorkflowInstance*, runner : WorkflowRunner*)
+update(task : HumanTask&) : void
AuditObserver
-registry : ParticipantRegistry*
+AuditObserver(subject : HumanTask*, instance : WorkflowInstance*, registry : ParticipantRegistry*)
+update(task : HumanTask&) : void
LeastLoadedAssignment
-registry : ParticipantRegistry*
+LeastLoadedAssignment(registry : ParticipantRegistry*)
+assign(role : const Role&) : Participant*
AssignmentRule
+assign(role : const Role&) : Participant*
RoleBasedAssignment
-registry : ParticipantRegistry*
-rules : AssignmentRule*
+RoleBasedAssignment(registry : ParticipantRegistry*, rules : AssignmentRule*)
+assign(role : const Role&) : Participant*
AutomatedTask
#executed : bool
+AutomatedTask(id : int, name : const string&)
+execute() : void
+getStatus() : string
+setStatus(operation : const string&) : void
+isClosed() : bool
+printTaskInfo() : void
HumanTask
-assigneeId : int
-dueDate : string
-currentState : TaskState*
-id : int
-observers : vector<TaskObserver*>
+HumanTask(id : int, name : const string&, requiredRole : const Role&, dueDate : const string&, assigneeId : int)
+assignedParticipant() : int
+getAssignee() : int
+getDueDate() : string
+getStatus() : string
+setStatus(operation : const string&) : void
+isClosed() : bool
+addObserver(observer : TaskObserver*) : void
+removeObserver() : void
+setPriority(priority : bool) : void
+attach(observer : TaskObserver*) : void
+detach(observer : TaskObserver*) : void
+notify() : void
+printTaskInfo() : void
HumanTaskCreator
-director : ProcessDirector
-builder : HumanTaskBuilder
+createTask(id : int, name : const string&, assigneeId : int, dueDat...
AutomatedTaskCreator
+createTask(id : int, name : const string&, assigneeId : int, dueDat...
TaskComposite
-children : vector<TaskComponent*>
+add(task : TaskComponent) : void
+remove(task : TaskComponent) : void
+getChild(index : int) : TaskComponent
+getAllTasks() : vector<TaskComponent*>
+printTaskInfo() : void
<<instance>>
RunOnceObserver
-done : vector<TaskComponent*>
-today : bool
+RunOnceObserver(basis : const vector<TaskComponent*>&)
+checkToday() : void
+update() : bool
+isDue() : TaskComponent*
PendingTaskHolder
-done : vector<TaskComponent*>
-today : bool
+PendingTaskHolder(instance : const WorkflowInstance&, participantId : int)
+checkNow() : bool
+pop() : TaskComponent*
TaskCreator
+createTask(id : int, name : const string&, assigneeId : int, dueDate : const string&, requiredRole : const Role&) : TaskComponent*
TaskStates
<<interface>>
+handle(task : HumanTask&, operation : const string&) : TaskState*
+getName() : string
+isClosed() : bool
PendingState
+handle(task : HumanTask&, operation : const string&) : TaskState*
+getName() : string
+isClosed() : bool
InProgressState
+handle(task : HumanTask&, operation : const string&) : TaskState*
+getName() : string
+isClosed() : bool
CompletedState
+handle(task : HumanTask&, operation : const string&) : TaskState*
+getName() : string
+isClosed() : bool
CancelledState
+handle(task : HumanTask&, operation : const string&) : TaskState*
+getName() : string
+isClosed() : bool
ApprovalRequest
-status : int
-requiredRole : Role
-stateLevel : int
-taskId : int
-assigneeTo : int
-rejected : bool
+ApprovalRequest(taskId : int, requiredRole : const Role&, requestLevel : int)
+getTaskId() : int
+getAssigned() : int
+getAssignedLevel() : int
+accept(participantId : int, level : int) : void
+reject(participantId : int) : void
+isAccepted() : bool
+isRejected() : bool
+canBeAccepted(byLevel : int) : bool
+isOpen() : bool
RequestHandler
-handlers : map<int, ApprovalHandler*>
+RequestHandler(rule : const Role&, rule : AssignmentRule*)
+handle(request : ApprovalRequest&) : void
ApprovalHandler
+setNextHandler()
+handle(request : ApprovalRequest&) : void